AGENTS.md — MINERVA QCELL P&ID (Colour-Line-First Pipeline)

Mission

Decompose the **real** QCELL / RFCELL P&ID SVG drawings into **colour-defined process lines**, preserving each line's sequence and scope/boundary context, so the lines can be isolated, reviewed, and reassembled into the full P&ID without losing engineering meaning. The success metric is "can we isolate each colour/process line and roll it back into the full P&ID" — **not** tag count.

Colour-first pipeline (the only supported flow)

1. **Ingest** — copy real sources into `data/svg/` (STOP if < 2 SVGs). PDF → `data/pdf/`, PPTX → `data/ppt/`.
2. **Extract** — `parser.extract_elements()` walks every drawable element and reads colour/style with **inline-style precedence**: `style.get(key) or elem.attrib.get(key)` (inline style wins). The effective process colour is the stroke if present, else the fill.
3. **Cluster** — `parser.classify_colour()` assigns each colour to the nearest canonical anchor by RGB distance (not exact hex):

Colour	Code	Meaning
BLUE / NAVY	A / A′	4.5 K main line + internal branch
CYAN / TEAL	B / B′	2 K internal line
GREEN	W	coupler (splits from BLUE A inside QM)
OLIVE	S	warm S line
GREY	V	vent line (per module, to outside)
RED / ORANGE	D / E	warm/cold manifold
BLACK	structure	boundary / symbols / unknown
other	unknown	unresolved (flagged, e.g. magenta)

1. **Model** — emit `colour_inventory.json`, `line_model.json`, and the seven per-colour files in `data/model/lines/`.
2. **Validate** — `reports/W002_colour_line_validation.md` + `navigation.json`.
3. **Publish** — `publish/colour_line_collage.html`: full drawing + one isolated view per colour line (A3 landscape, monochrome-safe labels).

Run

```
python3 src/abacus_svg_pid/cli.py            # build the colour line model
python3 src/abacus_svg_pid/render_collage.py # build the collage
python3 tests/test_colour_model.py           # assertions
```

Hard rules

- **Preserve sequence** — never geometrically re-order a line's components unless arrow / left-to-right evidence exists (that evidence is **W004**).
- **Claim ≠ Complete** — do not claim a colour is semantically correct unless it maps to the canonical table or it is explicitly listed as unresolved.
- Do **not** start UI, PPT, CI, package-rename, or the W003/W004 layer & geometry engine. Do not open a PR until success criteria 1–5 are non-zero.

Wave roadmap

See `configs/wave_registry.json` (W001–W009) and `reports/navigation.json` (current/next wave, status, SVGs found, blocking items). Current wave: **W005** (Tag & Instrument Register Reconciliation — XLSX coverage delta).

Governance — Claim ≠ Complete (binding)

These rules are binding on every wave. They exist because a "claim" of work is worthless to an engineering reviewer without verifiable evidence.

1. A wave is COMPLETE only if all four are true

A wave may be marked `complete` (in `configs/wave_registry.json` / `reports/wave_status.json`) **only when**:
1. **Output files exist** — the concrete artifacts named in `expected_output` are present on disk (or regenerable via `./make.sh` from tracked source).
2. **A validation report exists** — a `reports/W0NN_*.md` (or a section of the combined report) records what was checked and how.
3. **Runtime counts are recorded** — the actual numbers produced this run (elements, layers, pairs, components, …) are written down, not estimated.
4. **Known gaps are listed** — every unresolved item, low-confidence result, or carry-forward is named explicitly under `known_gaps`. An empty gap list must be a deliberate assertion, not an omission.

The normalized record of this lives in `reports/wave_status.json` with schema `{wave, claim, actual{…}, pass, known_gaps[]}`. `pass: true` requires all four conditions above. **Do not** flip a wave to complete to make a report look finished.

2. Never silently delete unresolved data

- Do **not** drop, prune, or "clean away" tags, text nodes, geometry elements, or colour codes just because they did not classify. Unresolved items are **flagged and counted** (e.g. `unknown`, `unmapped`, `floating`, `low_confidence`), never deleted.
- Reconciliation must balance: every input element is either mapped or appears in an explicit unresolved bucket. The layer sum-check (`drawable + text = total`, exact) is the canonical example — see the combined report.
- A smaller count is **not** evidence of progress. Losing elements between phases is a regression to be explained, not hidden.

3. Colour is legend-defined truth — but cross-validate it

- For **this** drawing, colour is the legend-defined ground truth for process line identity (per the canonical table above). That is the correct primary signal and we keep it primary.
- However colour must **not** be treated as the sole truth. A line's colour should be cross-checked against the **tag / instrument class** (ISA prefix,

see `configs/isa_classes.json`) and any nearby text. Where colour and tag-class disagree, the element is flagged for review — not force-resolved.

- This is a **documented principle**, not yet an automated gate. The cross-validation engine (colour-vs-tag-class agreement scoring) is future scope and must not be silently assumed to exist. Until then, disagreements are surfaced in validation reports, never auto-corrected.

4. Reproducibility over committed artifacts

- Derived outputs (`output_v6/`, `data/model/`, `data/pemo/`, `publish/`, `reports/*.xlsx`) are **regenerable**, not source. They are git-ignored and rebuilt by `./make.sh` from tracked source (`src/`, `segmentation/data/`, `configs/`, `data/svg/`). A reviewer must be able to clone, run `./make.sh`, and reproduce every number in the reports.
- Tracked source of record: `src/`, `segmentation/data/*.json`, `configs/`, `data/svg/`, `tests/`, `reports/*.md`, `docs/`.